Sections 21–22: Advanced Challenges & Learning Roadmap

21. Advanced Challenges (20 Blue-Team Scenarios)

Each is a self-contained mini-project once the core lab is functional. Do these **after** Sections 1-17 are working end-to-end — they assume a stable detection pipeline, not a substitute for building one.

#	Scenario	Category	What you build/detect
1	Ransomware simulation (safe, self-contained — e.g. a script that mass-renames/encrypts files in a test folder, no real ransomware binary)	Impact	Detect mass file modification rate via Sysmon Event ID 11 volume spike; write a rule triggering on >N file writes/sec from one process
2	Credential dumping via LSASS + registry hive extraction (`reg save hklm\sam`)	Credential Access	Extend 10.12 to cover offline SAM/SYSTEM hive extraction, not just live LSASS access
3	Lateral movement via PsExec/WMI across two victim VMs	Lateral Movement	Chain detections 10.13 + hunting exercise 11 across two hosts, build a "lateral movement path" timeline
4	Living-off-the-land (LOLBins): `certutil.exe` used to download a payload	Defense Evasion	Extend hunting exercise 3, write a Sigma rule specifically for `certutil -urlcache -f`

#	Scenario	Category	What you build/detect
5	Web application attack: SQLi against a deliberately vulnerable app (DVWA/Juice Shop, isolated)	Initial Access	Suricata signature tuning for SQLi patterns, correlate with web shell detection (10.16)
6	Insider threat simulation: authorized user exfiltrating data via USB/cloud upload	Exfiltration	Build a hunting query for large outbound transfers to personal cloud domains combined with USB device Event ID 6416
7	Phishing simulation: macro-enabled document spawning PowerShell	Initial Access	Full chain: 10.1 → 10.2 → 12.2 process tree exercise using a real (safe, self-written) macro test doc
8	Data exfiltration via DNS tunneling	Exfiltration	Full build-out of detection 10.14 with an actual `dnscat2` or `iodine` test (isolated network only)
9	Golden ticket / Kerberos abuse (requires AD — optional extension)	Credential Access	Deploy a minimal AD lab, detect via Event ID 4769 anomaly volume (hunting exercise 13)
10	Pass-the-hash lateral movement	Lateral Movement	Detect via anomalous Event ID 4624 LogonType 3 with NTLM, paired with no corresponding interactive logon

#	Scenario	Category	What you build/detect
11	Fileless malware (PowerShell reflective loading, no disk artifact)	Defense Evasion	Detect via Sysmon Event ID 7 (unusual DLL load) + Event ID 8 (CreateRemoteThread) correlation
12	Log tampering / anti-forensics (clearing Security event log)	Defense Evasion	Full build-out of hunting exercise 16, alert on Event ID 1102
13	Malicious scheduled task disguised as legitimate software update	Persistence	Extend 10.9, build a whitelist of legitimate update-task naming patterns to reduce FPs
14	Reverse shell over HTTPS (blends with normal traffic)	C2	Extend 10.15 to Zeek `ssl.log` — detect self-signed/unusual JA3 fingerprints on outbound 443
15	Brute force distributed across multiple source IPs (botnet-style)	Credential Access	Extend 10.5's per-source-IP threshold to a per-target-account aggregate across many IPs
16	Supply-chain style attack: malicious script in a downloaded "tool"	Initial Access	YARA rule authoring exercise (extend 10.17) against a benign-but-flagged test file
17	Privilege escalation via unquoted service path	Privilege Escalation	Detect via Sysmon Event ID 1 for services with unquoted

#	Scenario	Category	What you build/detect
			`ImagePath` containing spaces
18	Domain generation algorithm (DGA) C2 domains	C2	Extend 10.14's entropy detection specifically to DGA-pattern domains (statistical n-gram scoring)
19	Multi-stage attack chain (recon → initial access → persistence → credential access → lateral movement) run as one continuous exercise	Full chain	Produce a single incident report (12.9 template) covering the entire chain — your best portfolio artifact
20	Purple-team exercise: pick any MITRE technique not yet covered, write the Atomic Red Team test, then author the detection *before* running the test (true detection-engineering workflow, not detect-after-the-fact)	Any	New Sigma rule + coverage matrix update (Section 10 appendix)

Bonus 21 (perimeter — new, not counted in the "20" above for the same reason as file 05's bonus hunts): Threshold evasion against the Sophos deny-burst rule — scan slowly enough (below 15 attempts/60s) to stay under detection 10.19's frequency threshold, then document the gap this exposes: a fixed threshold is trivially evadable by anyone who reads your rule (or any public writeup like this one). Fix it by adding a second, longer-window/lower-threshold rule (e.g., 5 attempts/10 minutes) alongside the fast one — the same "layer multiple time windows" principle used in real detection engineering to catch both noisy and patient attackers. This is a strong "I found a limitation in my own detection and fixed it"

story for an interview, in the same spirit as the password-spraying gap already documented for detection 10.5.

---

22. Learning Roadmap (after completing this lab)

Immediate next skills (fill actual gaps this lab exposes):

1. **Active Directory security** — this lab uses standalone Windows; add a Domain Controller VM, learn Kerberos attack paths (Kerberoasting, Golden/Silver Ticket), BloodHound for AD attack-path mapping.
2. **EDR internals** — understand what a commercial EDR (CrowdStrike, SentinelOne, Defender for Endpoint) does that Sysmon+Wazuh doesn't (kernel callbacks, memory scanning, cloud correlation) — useful to *articulate* even without access to one.
3. **SOAR/automation** — take the manual investigation steps in Section 12 and script them (Python + Wazuh API) — this is the natural "future work" extension mentioned in Section 1.2.
4. **Cloud SOC** — replicate a reduced version of this lab in AWS/Azure (CloudTrail/Azure Activity Log → SIEM), since most real SOC roles are hybrid or cloud-native now.
5. **Malware analysis basics** — static/dynamic analysis of the Atomic Red Team payloads you already generated (safe, known-scope samples) — a natural bridge from detection engineering into DFIR.

Certifications that map directly onto what this lab teaches:

- CompTIA Security+ (foundational, if not already held)
- Blue Team Level 1 (BTL1) — near-exact overlap with this lab's scope
- GIAC GCIH / GCFA — if pursuing DFIR specifically
- eJPT / PNPT — offensive-side certs that make your *detection* engineering sharper (you write better detections once you understand the attacker's actual constraints)

Reading/practice to run in parallel, not after:

- MITRE ATT&CK Navigator — revisit your coverage matrix (Section 10 appendix) monthly, add techniques
- DetectionLab (Chris Long) — a more automated (Vagrant/Packer) version of a similar lab, good for comparing your manual build against an infrastructure-as-code approach
- SigmaHQ rule repository — read rules written by working detection engineers, not just your own

---

Project Complete — Repository Structure

SOC-Threat-Detection-Lab/

├── README.md (Sections 1-5: Overview, Architecture, Requirements, Network Design)

├── 02-Installation-and-SIEM-Setup.md (Sections 6-9)

├── 03-Detection-Engineering.md (Section 10 — 20 detections + Sigma/Wazuh rules)

├── 04-Threat-Simulation-and-Investigation.md (Sections 11-12)

├── 05-Threat-Hunting-and-Dashboards.md (Sections 13-14 — 20 hunts)

├── 06-Tuning-IR-Documentation.md (Sections 15-17)

├── 07-Advanced-Challenges-and-Roadmap.md (Sections 21-22)

├── 08-Sophos-Firewall-Integration.md (Perimeter layer — real firewall, syslog, decoders)

├── AUDIT-LOG.md (Full findings from the senior-review pass — keep this in your repo, it's a credibility asset, not a liability)

├── Architecture/ → diagram exports (PNG/draw.io)

├── Screenshots/ → evidence captures per Section 11.8 test table

├── Sigma/ → raw .yml Sigma rules from Section 10

├── Rules/ → converted Wazuh/Elastic-native rules

├── Dashboards/ → exported Kibana dashboard JSON

├── Logs/ → sample raw log excerpts (sanitized)

├── Reports/ → filled Project Report + Incident Reports (17.3, 17.7)

├── Runbooks/ → SOC Runbook, Detection Runbook (17.5, 17.6)

├── Threat-Hunting/ → filled hunt reports (17.8)

├── Detection-Engineering/ → per-rule quick-reference cards

└── Incident-Response/ → containment/eradication/recovery logs per incident

Resume Points (ATS-friendly, updated)

- Designed a three-tier security monitoring pipeline — Sophos Firewall (perimeter), Sysmon/Wazuh (endpoint), MISP (threat intel) — across 6 isolated VMs/devices simulating enterprise telemetry end-to-end.
- Configured Sophos Firewall syslog forwarding and authored custom Wazuh decoders/rules to parse and correlate perimeter firewall, IPS, and VPN events.
- Authored 20 MITRE ATT&CK-mapped detection rules (Sigma + Wazuh-native) covering credential access, persistence, lateral movement, C2, and perimeter reconnaissance/brute-force techniques.
- Integrated MISP threat intelligence with live abuse.ch feeds for automated IOC enrichment across both endpoint and perimeter alert sources.
- Validated detection coverage via Atomic Red Team and manual adversary simulation (Nmap, Hydra, Netcat, PowerShell); measured mean-time-to-alert and false-positive rate.
- Conducted 20 structured threat-hunting exercises using KQL and Sigma against Windows/Linux/perimeter telemetry.
- Performed a structured self-audit of the full pipeline (architecture, decoders, rules, scripts) catching and fixing decoder logic errors, a misconfigured field match, and a hardcoded-credential risk before deployment — documented in `AUDIT-LOG.md`.

Before publishing: your internship's offer letter (Rise Tech Software Pvt. Ltd.) explicitly requires confidentiality over "client credentials, firewall configurations, network topology, IP addressing schemes, security policies, or infrastructure-related information observed during training." None of the above bullets reference a client, and none should — every IP, hostname, and screenshot in this public repo must come from your own isolated lab, never from an actual client site you access during the 30-day placement. Keep the two worlds separate; it's both the ethical requirement and, practically, the only way this repo stays publishable without asking anyone's permission.

LinkedIn Post Draft

> I built a home SOC lab from the ground up — not a tutorial follow-along, an actual detect → investigate → enrich → respond pipeline, with a real firewall in front of it.
>
> Stack: Sophos Firewall at the perimeter, Wazuh SIEM + Elastic for correlation, Sysmon on the endpoint, MISP for threat intel enrichment against live abuse.ch feeds.

I configured Sophos to forward syslog to Wazuh, wrote the custom decoder to parse it, and authored 20 MITRE ATT&CK-mapped detections spanning perimeter brute force, endpoint credential dumping, and lateral movement — then validated every one with real attack simulation (Nmap, Hydra, Netcat, Atomic Red Team) and logged what actually fired vs. what didn't.

The most useful part wasn't the tools working — it was finding where they didn't. My first firewall-auth-failure rule matched the wrong log field entirely and would have silently never fired; my brute-force rule missed password spraying because it was keyed on failures-per-account, not failures-per-source-across-accounts. Finding and fixing those taught me more than any tutorial would have.

Full writeup, decoders, Sigma rules, and architecture diagrams on GitHub: [link]

#cybersecurity #soc #blueteam #detectionengineering #threatintelligence #sophos

---

This completes the full project documentation set (Sections 1-22). All files are organized to drop directly into a GitHub repository matching the structure above.